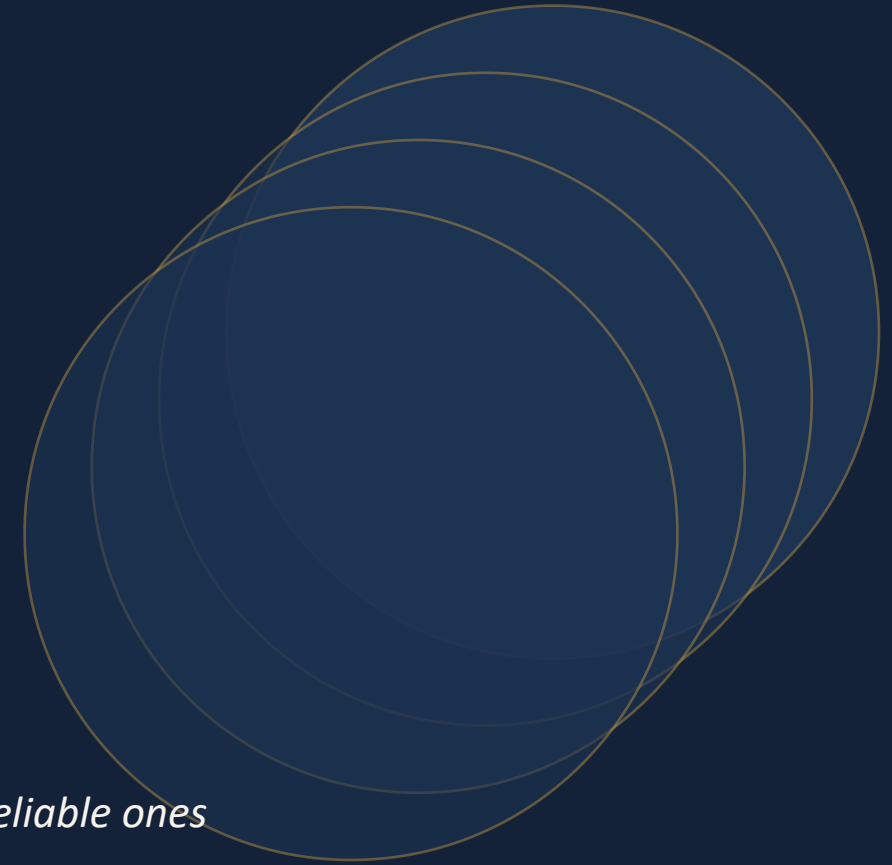


MACHINE LEARNING • ENSEMBLE METHODS

Random Forest

How an ensemble of decision trees turns noisy, overfit predictions into stable, reliable ones



Definition • 3-Step Mechanism • Strengths & Limits • Tuning Reference

Why Do We Need a Random Forest?

A Random Forest is an ensemble algorithm that builds a "forest" of many individual decision trees and combines their predictions into a single, more reliable output.

A Single Decision Tree

Suffers from high variance and overfitting

- If allowed to grow deep, it memorizes noisy data, outliers, and tiny quirks in the training set.
- It creates overly complex, jagged rules that fail on new, unseen test data.

Result: excellent on training data, unreliable on real-world data.

The Random Forest Solution

Trust many diverse trees, not one

Instead of one deep, overconfident tree, a Random Forest trains hundreds of diverse trees in parallel and takes their:

**Majority
Vote**

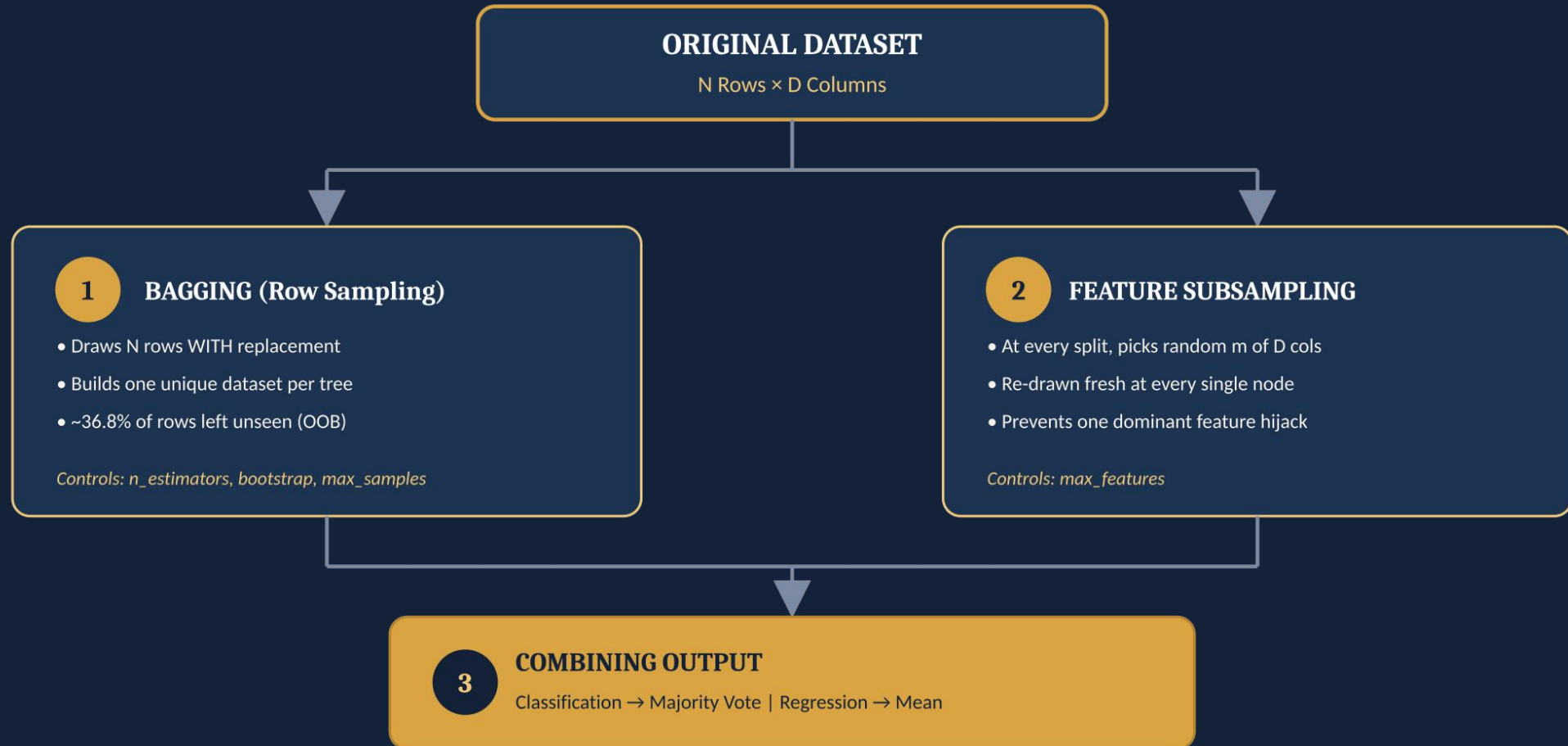
Classification

Average

Regression

Individual errors cancel each other out — producing a far more stable, reliable prediction.

The 3 Steps of a Random Forest



STEP 1

Bagging — Bootstrap Aggregating (Row Sampling)

Original Dataset (N rows)

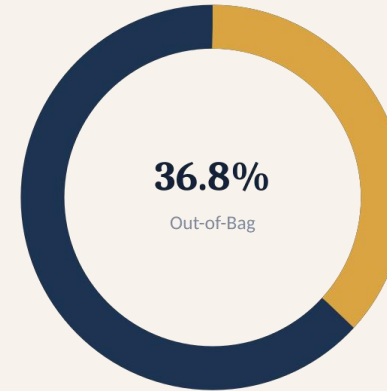
Row 1
Row 2
Row 3
Row 4
Row 5
Row 6
Row 7
Row 8

sample w/
replacement

Tree's Bootstrap Sample

Row 1
Row 3
Row 3
Row 4
Row 6
Row 6
Row 7
Row 8

Rows 2, 4, 8 never drawn → Out-of-Bag



■ OOB rows (~36.8%) — built-in validation set
■ In-bag rows (~63.2%) — used for training

Rows are drawn with replacement until the tree's dataset reaches size N — duplicates are expected, not an error.

Why OOB Matters

Because each tree never sees its ~36.8% Out-of-Bag rows, they act as a built-in cross-validation test — no separate test split needed.

Key Hyperparameters

n_estimators

total number of trees in the forest

bootstrap

True by default — enables sampling w/ replacement

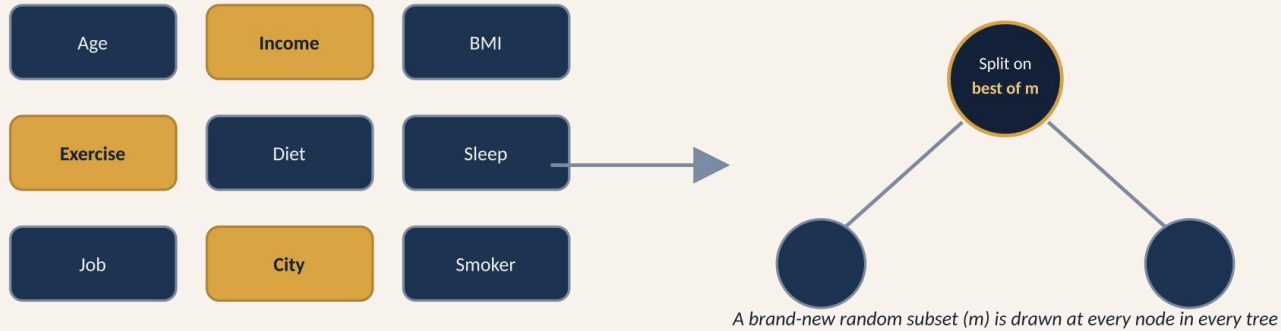
max_samples

fraction of rows drawn per tree (e.g. 0.8)

STEP 2

Feature Random Subsampling (Column Sampling)

All Features Available (D = 9)



Amber = randomly drawn subset (m) considered at this node split

Defaults for m (max_features)

Classification: $m = \sqrt{D}$ ($D=16 \rightarrow m=4$)

Regression: $m = D / 3$

Defaults for max_features (m)

Classification

$m = \sqrt{D}$ ($D=16 \rightarrow m=4$)

Regression

$m = D / 3$

Crucial Detail

Subset size (m) stays constant, but the specific features are freshly re-drawn at every single split, in every tree.

If every tree saw all features, dominant columns (e.g. Income) would win the root split every time — trees become nearly identical and the forest loses its diversity benefit.

STEP 3

Combining Predictions Across the Forest



Classification: Majority Vote

Votes across 5 trees:

Sick — 4 votes

Healthy — 1

Hard voting:

each tree casts one class vote

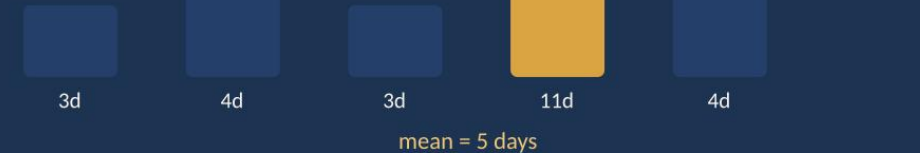
Soft voting:

tree probabilities are averaged

$$\hat{y} = \text{mode}(\hat{y}_1, \hat{y}_2, \dots, \hat{y}_B)$$

Regression: Arithmetic Mean

Predicted days across 5 trees:



$$\hat{y} = (1/B) \sum \hat{y}_b$$

Averaging smooths blocky single-tree
step predictions into a continuous curve

Combined-Output Errors & How to Fix Them

CLASSIFICATION — False Negative

Predicting "Healthy" for a Sick Patient

Why it happens

Class imbalance: 9,000 Healthy vs 1,000 Sick biases trees toward "Healthy."

Feature overlap: the sick patient's features resemble typical healthy values.

How to fix

Lower the decision threshold (e.g. flag "Sick" if $\geq 20\%$ of trees vote Sick) or set `class_weight='balanced'`.

REGRESSION — Overestimation

Predicting 11 Days for a 2-Day Normal Patient

Why it happens

Outliers in training: extreme cases (e.g. 60-day recovery) sharing features pull up leaf averages.

Right-skewed target: high extreme values pull MSE-based leaf means higher.

How to fix

Increase `min_samples_leaf` (e.g. to 10) to dilute outliers, or log-transform the target: `log(y)` before training.

What Data It Uses Well — and What It Avoids

Operates Well On

Complex, non-linear tabular data

models feature interactions like IF Age>50 AND Exercise<1

Mixed data types

handles numeric and categorical features cleanly

No scaling required

split thresholds ignore monotonic scaling; encoding is still required for text labels

Struggles With

Scenario	Why It Struggles	Alternative
Unstructured data (images, audio, text)	Flattening pixels/word sequences destroys spatial/sequential structure	CNNs / Transformers
High-dimensional sparse data (50,000+ columns)	$\sqrt{50,000} \approx 223$ random features often yields all zeros — useless splits	Linear SVM / Lasso / Logistic Regression
Time-series / trended data	Cannot extrapolate a trend beyond its max trained leaf value	ARIMA / Prophet / LSTM

What Random Forest Can & Cannot Do

CAN DO

■ Handle missing values & outliers well

isolated into deep, narrow branches and excluded from ~36.8% of trees by bagging

■ Provide built-in feature importance

measures how much each feature reduces impurity across the forest

■ Reduce overfitting

combining diverse trees drastically lowers variance vs. a single tree

CANNOT DO

■ Cannot extrapolate beyond training limits

if max training income is \$100k, it can never predict \$120k for a new case

■ Memory heavy & slow real-time inference

500 deep trees require substantial RAM and slower production predictions

■ Black box — loses interpretability

explaining 500 trees voting together is nearly impossible for stakeholders

Hyperparameter Tuning at a Glance

Optimized via GridSearchCV or RandomizedSearchCV

Hyperparameter	Scope	What It Controls	Tuning Direction
n_estimators	Forest	Number of trees in the forest	Higher is better, until it plateaus
max_features	Node	Random features (m) evaluated per split	Lower → more diversity; higher → stronger but more correlated trees
max_samples	Forest	Fraction of rows drawn per bootstrap	Lower speeds up training on huge datasets
max_depth	Tree	Maximum vertical levels per tree	Cap (e.g. 10–20) to prevent overfitting
min_samples_split	Tree	Min samples to attempt a split	Increase to stop splitting on noise
min_samples_leaf	Tree	Min samples per terminal leaf	Increase (5–10) to dilute outliers

Single Decision Tree vs. Random Forest

Property	Single Decision Tree	Random Forest
Structure	1 single decision tree	Ensemble of N decision trees
Variance / Overfitting	High variance, overfits easily	Low variance — voting/averaging cancels noise
Feature Selection	Evaluates all features at every split	Random subset (m) at every split node
Out-of-Bag Testing	Not applicable	Built-in test set (~36.8% of data/tree)
Extrapolation	Cannot extrapolate	Cannot extrapolate
Interpretability	High — visualizable flowchart	Low — ensemble "black box"

Bottom line: a Random Forest trades some interpretability for substantially better generalization and stability.